Spectral

January 12, 2026

0.0.1 B.5. Implement the spectral clustering algorithm and test its performance on the spiral dataset. Compare these clustering results with those obtained by the simple K-means clustering algorithm.

1 Spectral Clustering

Spectral clustering is a **graph-based clustering method** that uses linear algebra (eigenvalues and eigenvectors) to identify structure in data, especially when clusters are **non-convex** or not well separated in Euclidean space.

1.1 1. Intuition (Big Picture)

Instead of asking:

> *Which points are close in Euclidean space?*

Spectral clustering asks:

> **If we build a graph where similar points are strongly connected, how can we partition this graph into well-connected groups?**

Key idea: - Data points are treated as **nodes** - Similarity between points becomes an **edge weight** - Good clusters have **strong internal connections** and **weak external connections**

This transforms clustering into a **graph partitioning problem**.

1.2 Full Algorithm (Normalized Spectral Clustering)

1. Compute the similarity matrix W
2. Compute the degree matrix D
3. Compute the normalized Laplacian

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$$

4. Compute the first k eigenvectors of L_{sym}
5. Normalize each row of the eigenvector matrix
6. Apply k-means to the rows
7. Output cluster labels

1.3 Why Spectral Clustering Works

Spectral clustering approximately minimizes graph partitioning objectives such as: - **Ratio Cut** - **Normalized Cut**

The eigenvectors of the graph Laplacian reveal the natural low-dimensional structure of the data, making clusters more separable.

1.4 Geometric Interpretation

Spectral clustering is effective when data lies on a **non-linear manifold**.

For example: - k-means fails on the *two-moons* dataset - Spectral clustering succeeds by using graph connectivity instead of raw distance

---

1.5 When to Use Spectral Clustering

Advantages - Handles non-convex clusters - Works well on graph and network data - Effective for image segmentation and community detection

Limitations - Computationally expensive for large datasets - Requires careful construction of the similarity graph - Number of clusters k must be specified in advance

[4]:
```
# Standard libraries
import numpy as np
import pandas as pd
import seaborn as sns

# Plotting libraries
import matplotlib.pyplot as plt
import networkx as nx
from matplotlib import cm

# scikit-learn
from sklearn.datasets import make_blobs
from sklearn.datasets import make_circles
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

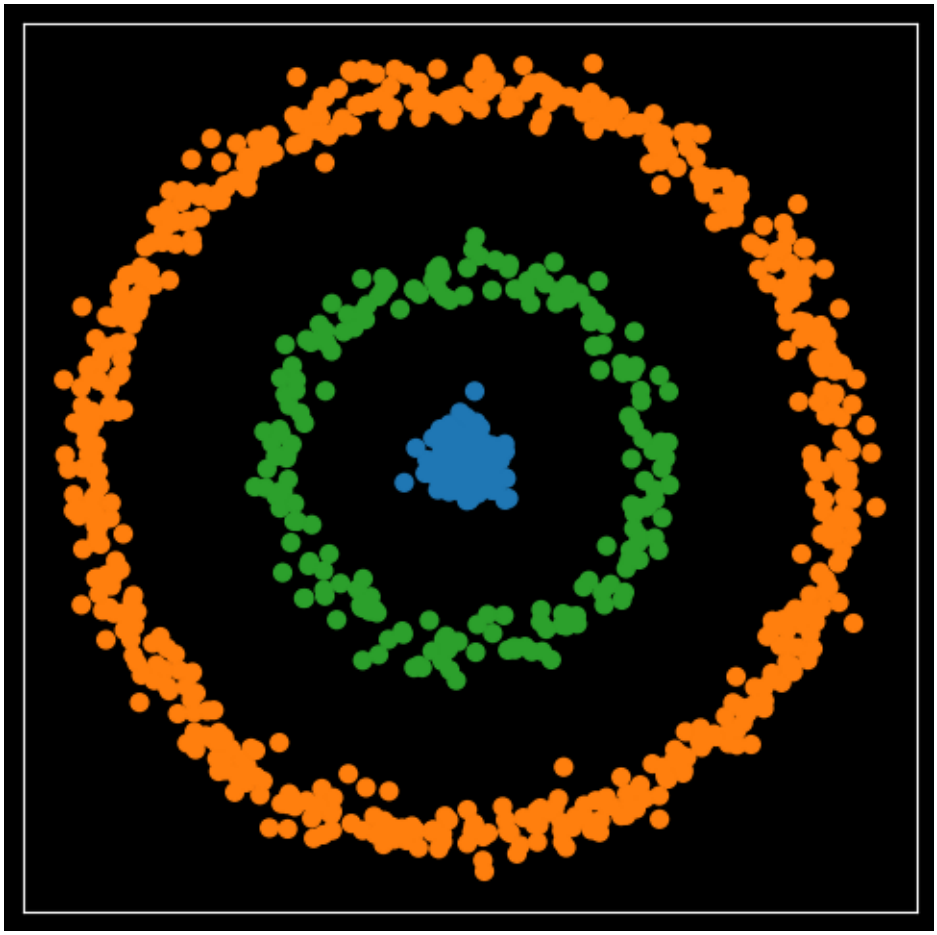
1.5.1 Data Preparation

[5]:
```
random_seed = 42
plt.style.use('dark_background')

X, y = make_blobs(n_samples=200, n_features=2, centers=1, cluster_std=0.15,␣
 ↪random_state=random_seed)
# center at the origin
```

```
X = X - np.mean(X, axis=0)

X1, y1 = make_circles(n_samples=(600, 200), noise=0.04, factor=0.5,␣
 ↪random_state=random_seed)
# add 1 to (make_circles) labels to account for (make_blobs) label
y1 = y1 + 1
# increase the radius
X1 = X1*3

X = np.concatenate((X, X1), axis=0)
y = np.concatenate((y, y1), axis=0)
```

[6]:
```
plot_colors = cm.tab10.colors
y_colors = np.array(plot_colors)[y]
```

[7]:
```
fig, ax = plt.subplots(figsize=(6,6))
ax.scatter(X[:,0], X[:,1], marker='o', s=40, color=y_colors)

ax.
 ↪tick_params(axis='both',which='both',bottom=False,top=False,left=False,right=False,
            labelbottom=False,labeltop=False,labelleft=False,labelright=False);
ax.set(xlabel=None, ylabel=None)
plt.show()
# plt.axis('off')
# plt.savefig('plot.png', bbox_inches='tight', dpi=600, transparent=True)
```



1.6 2. Step 1: Build a Similarity Graph

Given data points

$$x_1, x_2, \ldots, x_n$$

we construct a weighted graph.

1.6.1 Common similarity functions

Gaussian (RBF) kernel:

$$W_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

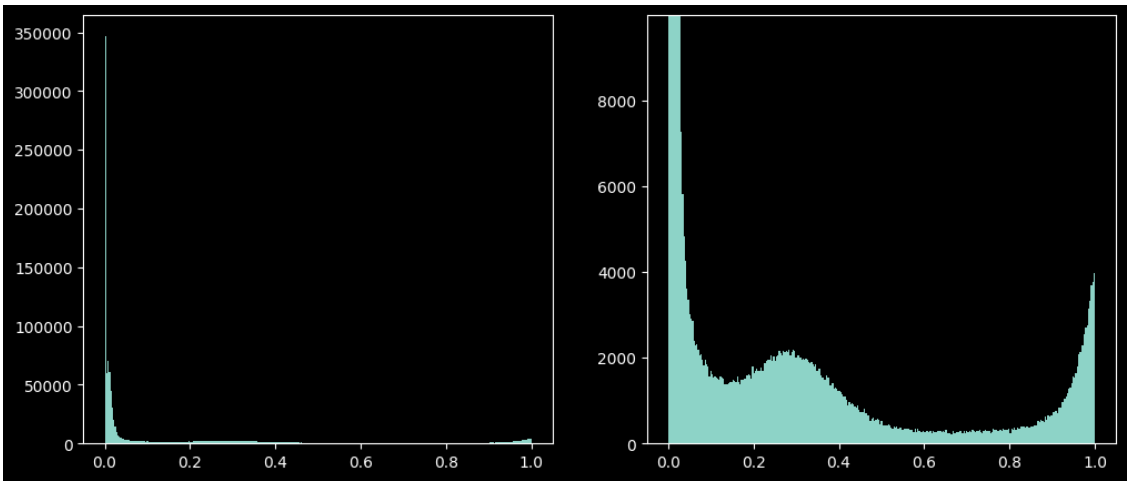
Other options: - k-nearest neighbors graph - ε-neighborhood graph - Cosine similarity (for text data)

This produces a **similarity (weight) matrix**:

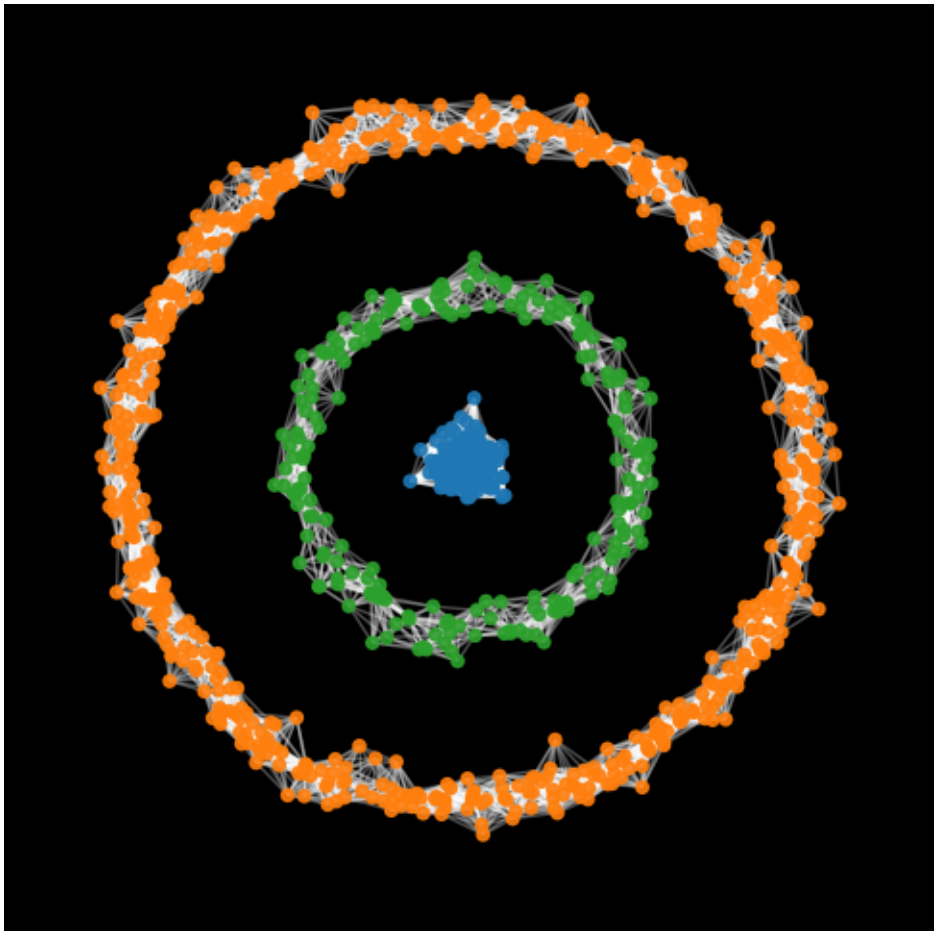
$$W \in \mathbb{R}^{n \times n}$$

[ ]:
```
sigma = 1
A = -1 * np.square(X[:, None, :] - X[None, :, :]).sum(axis=-1)
A = np.exp(A / (2* sigma**2))
np.fill_diagonal(A, 0)
```

[9]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

bin_values, _, _ = ax1.hist(A.flatten(), bins='auto')

# limit the y-axis to the value of the 10th bin
ax2.hist(A.flatten(), bins='auto')
ax2.set_ylim([0, bin_values[8]])

plt.show()
```



[10]:
```
A1 = np.copy(A)
A1[A1 < 0.9] = 0
G = nx.from_numpy_array(A1)

plt.figure(figsize=(6,6))
plt.axis('off')
nx.draw_networkx_nodes(G, pos=X, node_size=20, node_color=y_colors, alpha=0.9)
nx.draw_networkx_edges(G, pos=X, edge_color="white", alpha=0.3)
plt.show()
```



1.7 3. Step 2: Graph Laplacian

1.7.1 Degree matrix

$$D_{ii} = \sum_{j=1}^{n} W_{ij}$$

1.7.2 Types of graph Laplacians

1. **Unnormalized Laplacian**

$$L = D - W$$

2. **Normalized (symmetric) Laplacian**

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}$$

3. **Random-walk Laplacian**

$$L_{\text{rw}} = I - D^{-1} W$$

The graph Laplacian encodes the connectivity structure of the graph.

For this we are Using **Normalized (symmetric) Laplacian**

[20]:
```
# identity matrix
I = np.zeros_like(A)
np.fill_diagonal(I, 1)

# degree matrix
D = np.zeros_like(A)
np.fill_diagonal(D, np.sum(A,axis=1))
D_inv_sqrt = np.linalg.inv(np.sqrt(D))

L = I - np.dot(D_inv_sqrt, A).dot(D_inv_sqrt)
```

1.8 4. Step 3: Eigenvector Embedding

We solve the eigenvalue problem:

$$L_{\text{sym}} v = \lambda v$$

Let

$$v_1, v_2, \dots, v_k$$

be the eigenvectors corresponding to the **smallest** k **eigenvalues**.

Construct the matrix:

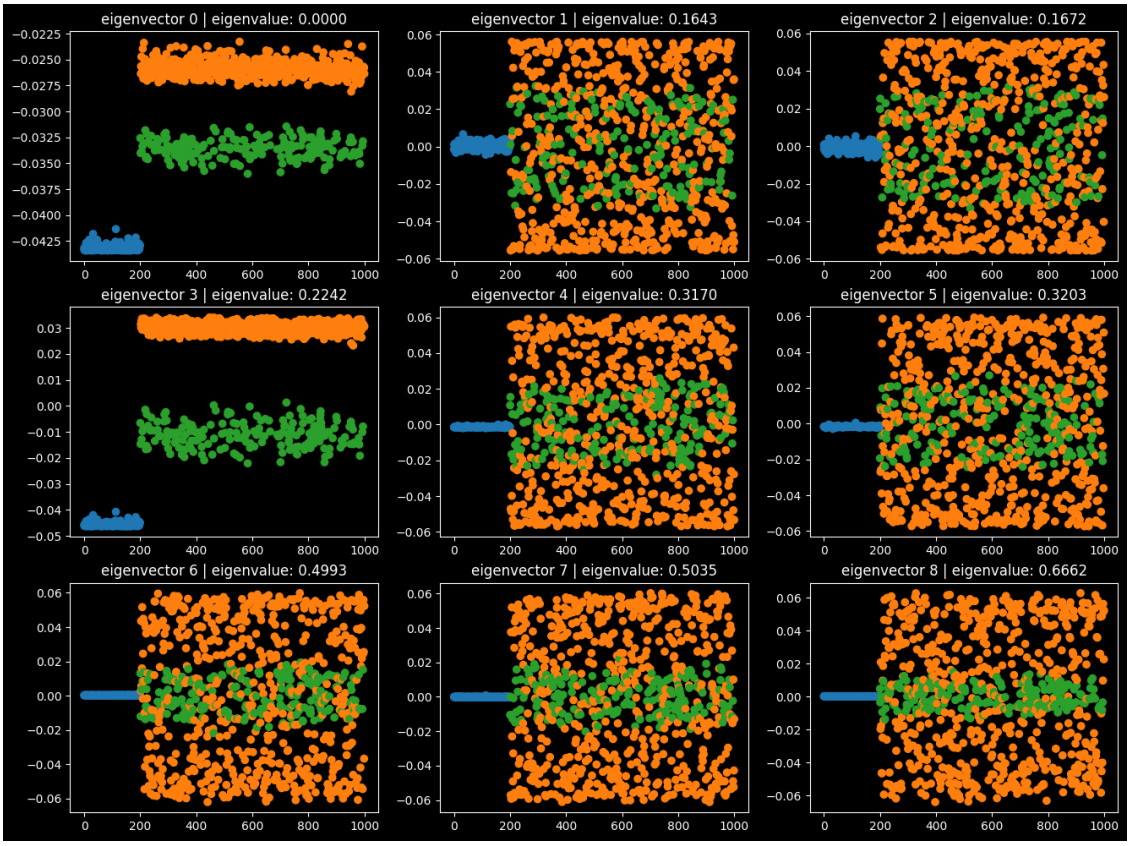
$$U = \begin{bmatrix} v_1 & v_2 & \cdots & v_k \end{bmatrix} \in \mathbb{R}^{n \times k}$$

Each row of U represents a new embedding of a data point.

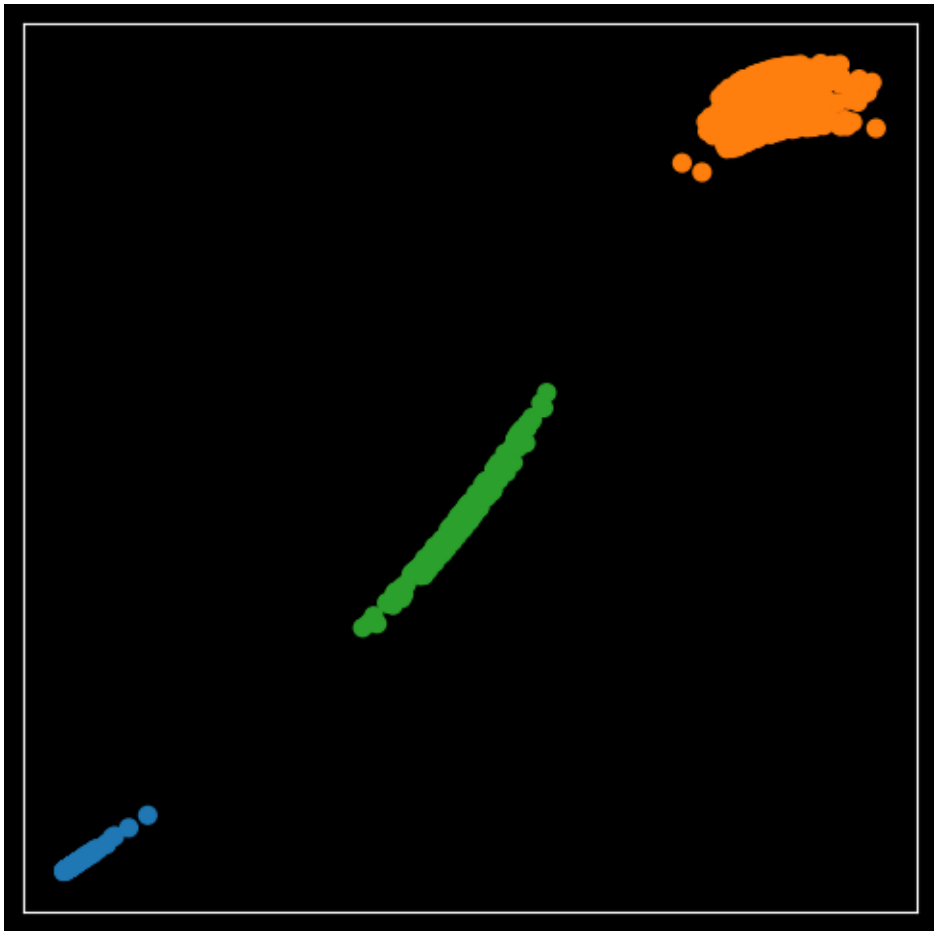
[12]:
```
eigenvalues, eigenvectors = np.linalg.eig(L)
eigenvalues = eigenvalues.real
eigenvectors = eigenvectors.real

# Order the eigenvalues in an increasing order
ind = np.argsort(eigenvalues, axis=0)
eigenvalues_sorted = np.take_along_axis(eigenvalues, ind, axis=0)

# Order the eigenvectors based on the magnitude of their corresponding␣
 ↪eigenvalues
eigenvectors_sorted = eigenvectors.take(ind, axis=1)
```

[13]:
```
fig, axs = plt.subplots(3, 3, figsize=(16, 12))
eigen_v_x = np.linspace(0, eigenvectors_sorted.shape[0], eigenvectors_sorted.
 ↪shape[0])

for j, ax in enumerate(fig.axes):
  eigen_v_y = eigenvectors_sorted[:,j]
```

```
  ax.scatter(eigen_v_x, eigen_v_y, marker='o', color=y_colors)
  ax.set_title(f'eigenvector {j} | eigenvalue: {eigenvalues_sorted[j]:.4f}')

plt.show()
```



[14]:
```
fig, ax = plt.subplots(figsize=(6,6))
ax.scatter(eigenvectors_sorted[:,0], eigenvectors_sorted[:,3], marker='o',␣
  ↪s=40, color=y_colors)

ax.
  ↪tick_params(axis='both',which='both',bottom=False,top=False,left=False,right=False,
            labelbottom=False,labeltop=False,labelleft=False,labelright=False);
ax.set(xlabel=None, ylabel=None)
plt.show()
```



1.9 5. Step 4: Clustering in Eigenvector Space

Treat each row of U as a point in:

$$\mathbb{R}^k$$

and apply **k-means clustering**.

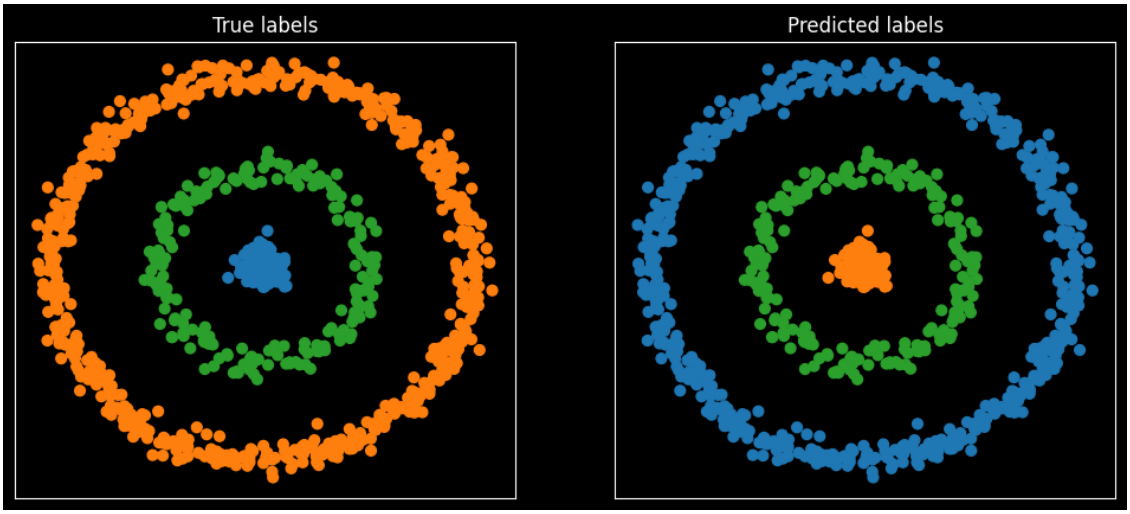
The resulting labels are the final cluster assignments.

[29]:
```
X_transformed = eigenvectors_sorted[:,[0,3]]

scaler = StandardScaler()
scaler.fit(X_transformed)
X_transformed_scaled = scaler.transform(X_transformed)


kmeans = KMeans(n_clusters = 3, random_state = random_seed, n_init='auto')
kmeans.fit(X_transformed_scaled)
```

[29]:
```
KMeans(n_clusters=3, random_state=42)
```

1.9.1 Data Plotting and Comparison with Actual cluster

[30]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

ax1.scatter(X[:,0], X[:,1], marker='o', s=40, color=y_colors)
ax1.
 ↪tick_params(axis='both',which='both',bottom=False,top=False,left=False,right=False,
           labelbottom=False,labeltop=False,labelleft=False,labelright=False);
ax1.set(xlabel=None, ylabel=None)
ax1.set_title('True labels')

ax2.scatter(X[:,0], X[:,1], marker='o', s=40, color=np.
 ↪array(plot_colors)[kmeans.labels_])
ax2.
 ↪tick_params(axis='both',which='both',bottom=False,top=False,left=False,right=False,
           labelbottom=False,labeltop=False,labelleft=False,labelright=False);
ax2.set(xlabel=None, ylabel=None)
ax2.set_title('Predicted labels')

plt.show()
```

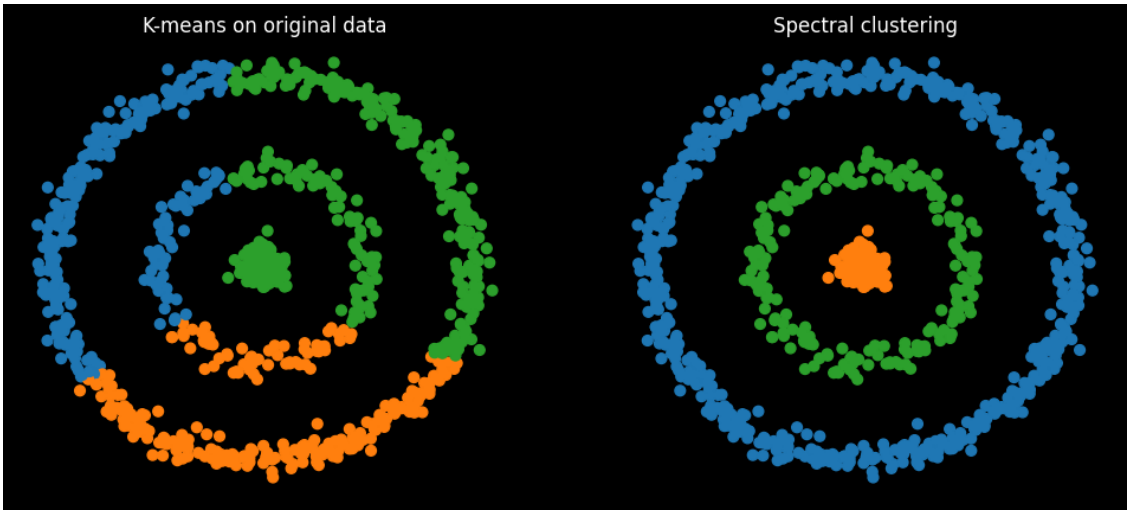


1.9.2 Comparison with Simple K-Means Clustering

[31]:
```
kmeans_raw = KMeans(n_clusters=3, random_state=random_seed, n_init='auto')
kmeans_raw.fit(X)
```

[31]:
```
KMeans(n_clusters=3, random_state=42)
```

[32]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,5))

ax1.scatter(X[:,0], X[:,1], s=40,
            color=np.array(plot_colors)[kmeans_raw.labels_])
ax1.set_title("K-means on original data")
ax1.axis('off')

ax2.scatter(X[:,0], X[:,1], s=40,
            color=np.array(plot_colors)[kmeans.labels_])
ax2.set_title("Spectral clustering")
ax2.axis('off')

plt.show()
```



1.10 Conclusion

K-means clustering fails to correctly separate the non-linear structure of the data, as it assumes convex clusters. Spectral clustering, by leveraging graph connectivity and Laplacian eigenvectors, successfully separates the data according to its intrinsic geometry.